

Image Recognition



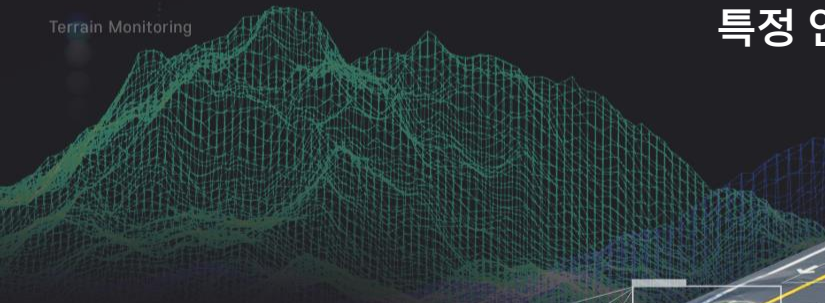
강원혁신플랫폼

컴퓨터 비전



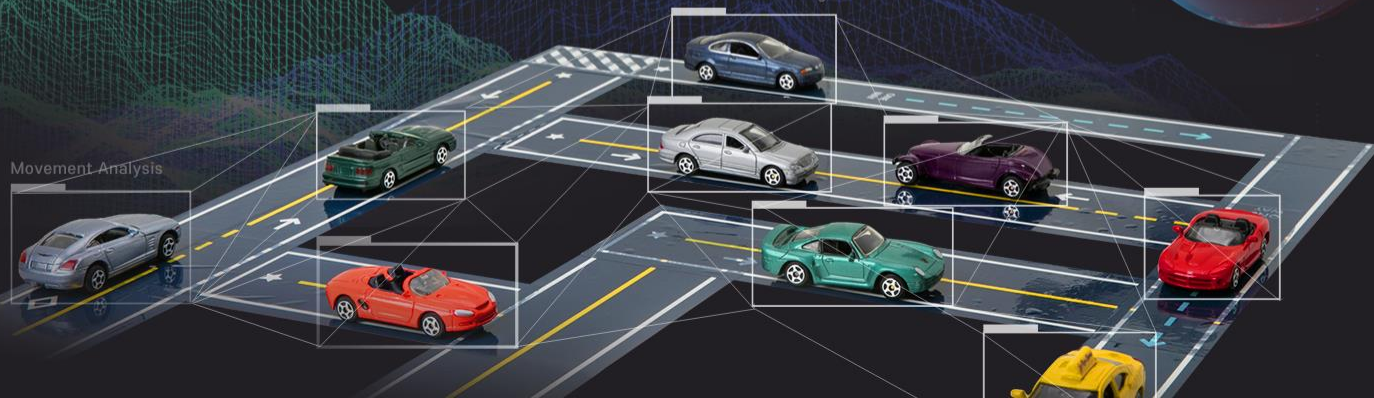
특정 인물 얼굴 인식(비디오)

Terrain Monitoring



Autonomous Driving

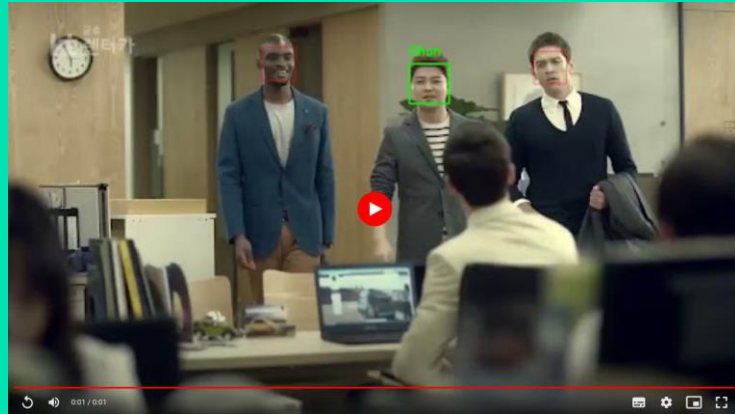
Movement Analysis



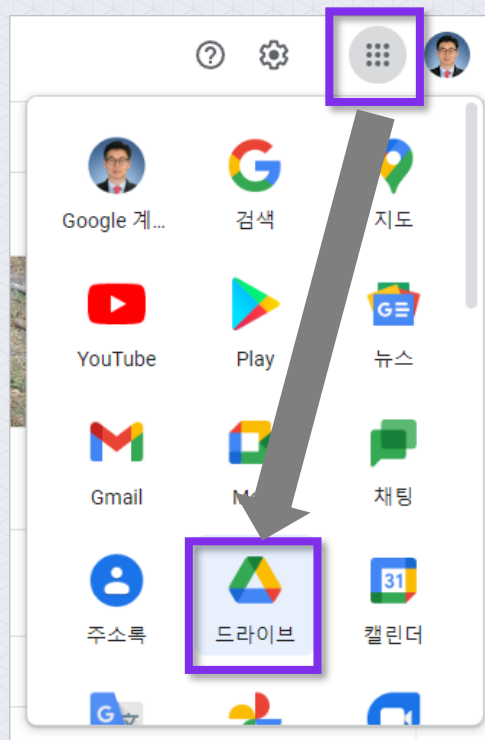
동영상에서 얼굴 인식을 수행해 보자

- 앞에서 전현무 방송인의 얼굴 이미지 사진에서 얼굴 특성값과 이름을 매칭하여 'encodings.picke' 파일 이름으로 저장해 두었음

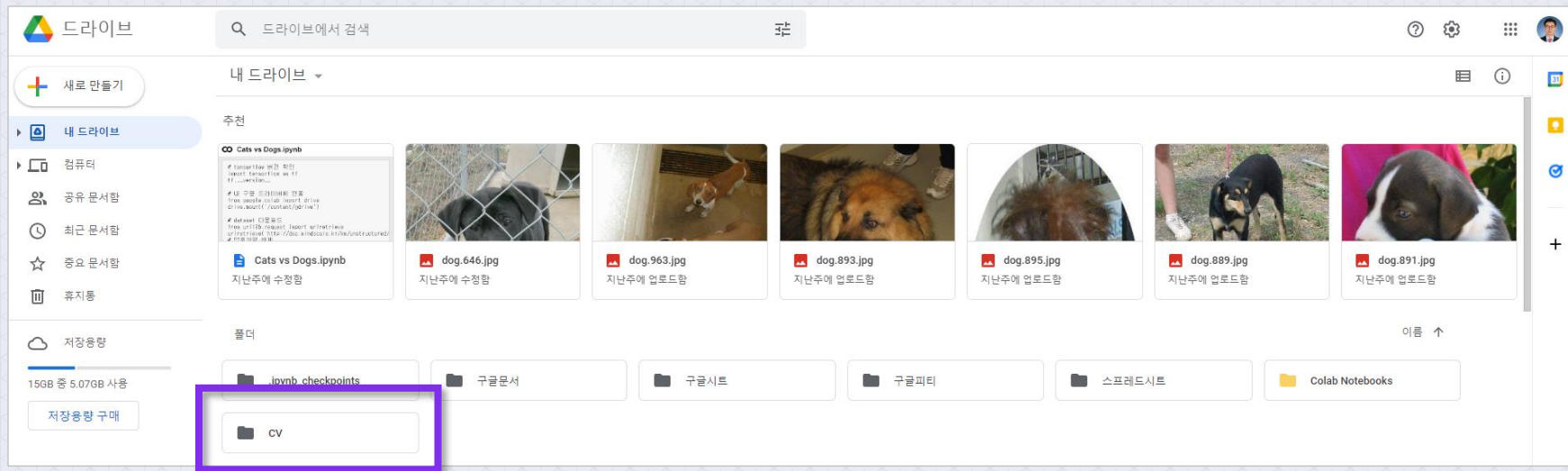
이번에는 동영상에서 전현무 얼굴 인식을 수행해 보자



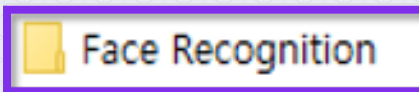
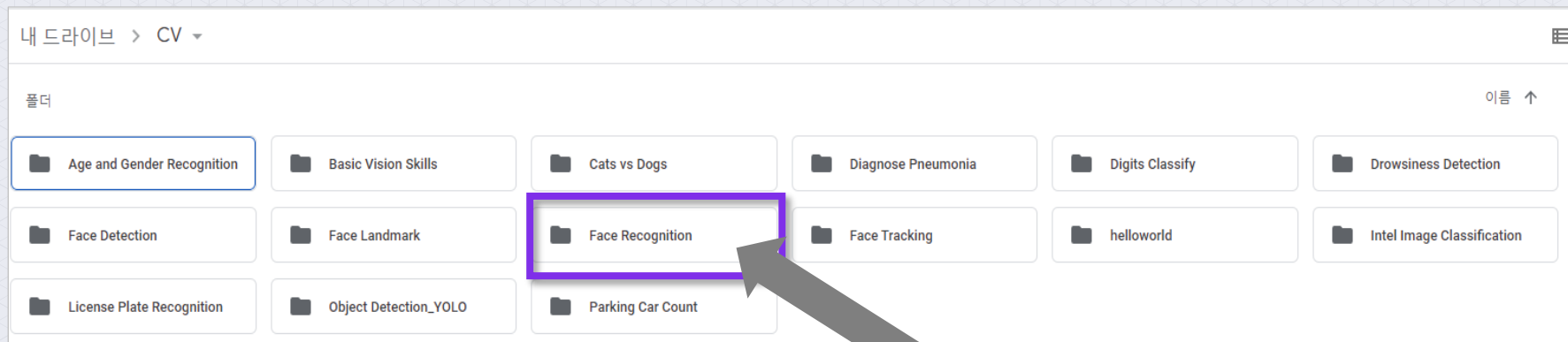
- 먼저, 구글 Colab을 이용해 보자
- 구글 크롬 브라우저에서 아래 그림과 같이 **구글 드라이브**를 클릭



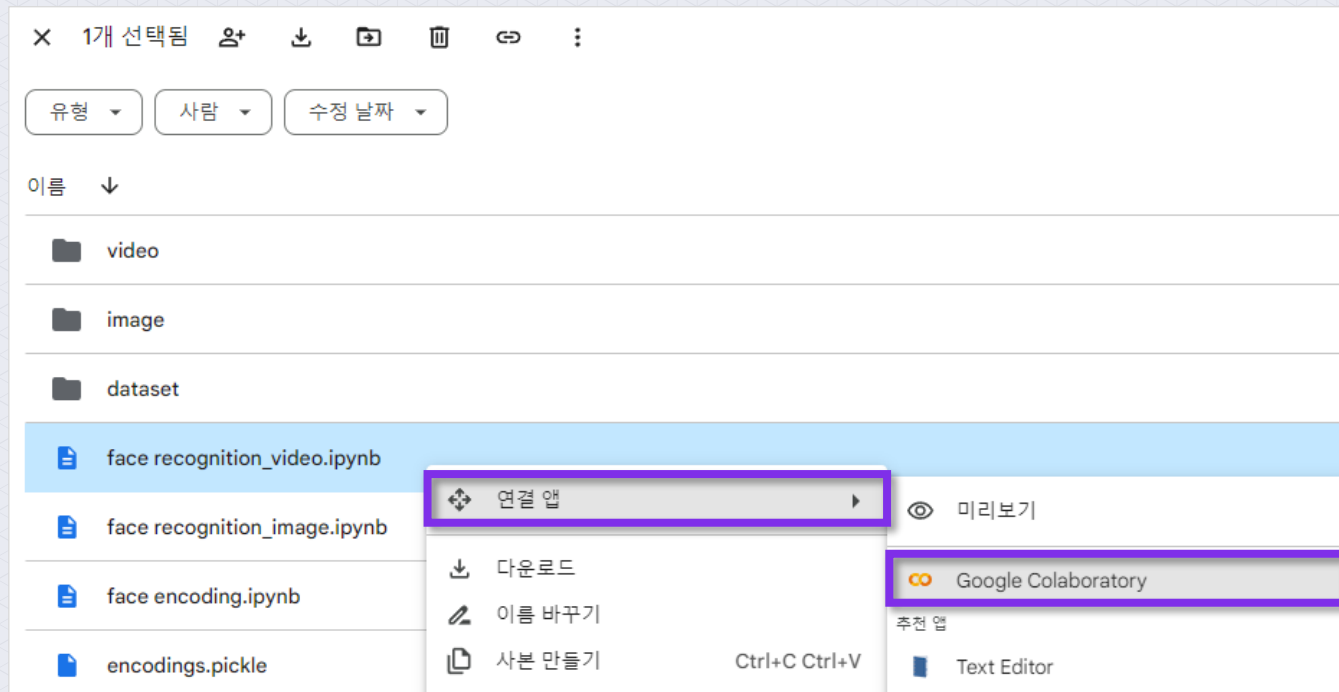
구글 드라이브에서 CV 폴더를 클릭



구글 드라이브 > CV > Face Recognition 폴더를 클릭



- ❶ 아래의 그림의 image 폴더에서 **face_recognition_video.ipynb**를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭



구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것임



The screenshot shows a Google Colab notebook titled "face_recognition_video.ipynb". The interface includes a left sidebar with icons for file management, search, and execution. The main area displays a code cell with the following content:

```
[1] # 내 구글 드라이브에 연동
    from google.colab import drive
    drive.mount('/content/gdrive')

Mounted at /content/gdrive

[ ] # 구글 Colab에서는 기본적으로 cmake 패키지가 설치되어 있어 따로 설치가 필요 없다.

    # dlib 패키지는 C++로 개발된 패키지이므로 컴파일 하기 위해서 cmake 패키지도 설치한다.
    # cmake install

    !pip3 install cmake
```

The first code block is marked as executed with a green checkmark and a "30 초" (30 seconds) duration. The output shows "Mounted at /content/gdrive". The second code block is currently empty, indicated by "[]".

- ① 내 구글 드라이브에 연동
- ② 구글 Colab 노트북 화면에서 실행 버튼 클릭



- ③ [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동
- ④ gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있음

- [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용]
- 아래 화면을 참고하여 **gdrive**에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

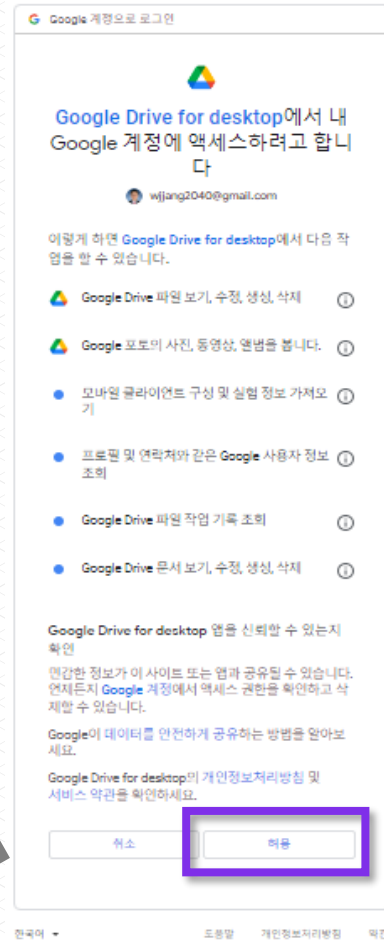
이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니오

Google Drive에 연결



구글 드라이버 접근 허용



■ face_recognition 패키지의 상세정보 버전을 확인해 보자

- 🔄 확인 결과 'face_recognition' 패키지가 설치되지 않은 것을 볼 수 있음

```
1 # 'face_recognition' 모듈(라이브러리, 패키지)의 상세정보 버전 확인
2
3 !pip show face_recognition
```

```
⚠ WARNING: Package(s) not found: face_recognition
```

■ face_recognition 패키지가 잘 설치된 것을 확인할 수 있음

```
1 # face_recognition 패키지는 구글 Colab에서 기본으로 지원하지 않아 반드시 설치해야 한다.  
2  
3 !pip3 install face_recognition          # face_recognition install  
4  
5 # !pip3 uninstall face_recognition      # face_recognition uninstall
```

```
Collecting face_recognition  
  Downloading face_recognition-1.3.0-py2.py3-none-any.whl (15 kB)  
Collecting face-recognition-models>=0.3.0 (from face_recognition)  
  Downloading face_recognition_models-0.3.0.tar.gz (100.1 MB)  
----- 100.1/100.1 MB 11.7 MB/s eta 0:00:00  
Preparing metadata (setup.py) ... done  
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.10/dist-packages (from face_recognition) (8.1.6)  
Requirement already satisfied: dlib>=19.7 in /usr/local/lib/python3.10/dist-packages (from face_recognition) (19.24.2)  
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from face_recognition) (1.22.4)  
Requirement already satisfied: Pillow in /usr/local/lib/python3.10/dist-packages (from face_recognition) (9.4.0)  
Building wheels for collected packages: face-recognition-models  
  Building wheel for face-recognition-models (setup.py) ... done  
  Created wheel for face-recognition-models: filename=face_recognition_models-0.3.0-py2.py3-none-any.whl size=100566169  
  Stored in directory: /root/.cache/pip/wheels/7a/eb/cf/e9ced74122b679557f597bb7c8e4c739cfac526db1fd523d  
Successfully built face-recognition-models  
Installing collected packages: face-recognition-models, face_recognition  
Successfully installed face-recognition-models-0.3.0 face_recognition-1.3.0
```

- 필요한 패키지와 모듈을 불러옴
- 구글 Colab 노트북 화면에서 실행 버튼 클릭



필요한 패키지와 모듈을 불러옴

```
import cv2
import face_recognition
import dlib
import pickle
import time
import io
import base64
from IPython.display import HTML

print('cv2 version :', cv2.__version__)
print('dlib version :', dlib.__version__)
```



```
cv2 version : 4.6.0
dlib version : 19.24.0
```

원본 동영상과 얼굴 특성값 파일 위치, 이미지에서 주요 특징을 캡처하는 표현 방법을 “cnn”으로 설정

주요 특징 캡처하는 시간
: 조금 느림

정확도 : 높음

```

1  # 원본 동영상과 얼굴 특성값 저장 위치 설정
2  # - 이미지에서 주요 특징을 캡처하는 표현 방법 : cnn
3  # - cnn : 조금 느리지만 정확도가 높음
4
5  file_name1 = 'gdrive/My Drive/CV/Face Recognition/video/chun_video1.mp4' # 원본 동영상
6  file_name2 = 'gdrive/My Drive/CV/Face Recognition/video/chun_video2.mp4' # 원본 동영상
7  encoding_file = 'gdrive/My Drive/CV/Face Recognition/encodings.pickle' # 저장해 두었던 특성 파일
8  unknown_name = 'Unknown' # 이름을 모를 경우 Unknown 으로 지정
9  output_name1 = 'output_cnn_chun_video1.mp4' # Detection 된 output 동영상
10 output_name2 = 'output_cnn_chun_video2.mp4' # Detection 된 output 동영상
11
12 # Either cnn or hog. The CNN method is more accurate but slower. HOG is faster but less accurate.
13 model_method = 'cnn'
    
```

■ 원본 동영상(1)을 출력

```
1 # 원본 동영상(1) Detection 하기 전에 동영상을 Display 한다.  
2  
3 video = io.open('gdrive/My Drive/CV/Face Recognition/video/chun_video1.mp4', 'r+b').read()  
4 encoded = base64.b64encode(video)  
5 HTML(data='''<video width="30%" controls>  
6 | | | | | <source src="data:video/mp4;base64,{0}" type="video/mp4"/>  
7 | | | | | </video>'''.format(encoded.decode('ascii')))
```



■ 원본 동영상(2)을 출력

```
1 # 원본 동영상(2) Detection 하기 전에 동영상을 Display 한다.  
2  
3 video = io.open('gdrive/My Drive/CV/Face Recognition/video/chun_video2.mp4', 'r+b').read()  
4 encoded = base64.b64encode(video)  
5 HTML(data='''<video width="30%" controls>  
6 |<source src="data:video/mp4;base64,{0}" type="video/mp4"/>  
7 |</video>'''.format(encoded.decode('ascii')))
```




```

1 # 얼굴 인식 및 결과 출력하는 함수
2 def detectAndDisplay(image, output_name):
3     start_time = time.time() # 시작 시간
4     # 이미지를 face_recognition 에서 처리하기 위해 RGB 형식으로 변경
5     rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
6
7     # 얼굴을 detection 하고 boxing 한다.
8     boxes = face_recognition.face_locations(rgb, model=model_method)
9
10    # 얼굴 영역(boxing Area)만 encoding(특성값)을 계산한다.
11    encodings = face_recognition.face_encodings(rgb, boxes)
12
13    # detection 된 얼굴의 이름을 저장할 배열
14    names = []
15
16    for encoding in encodings:
17        # boxing 된 특성이 pickle 파일의 encodings(특성값) 과 매칭되는 것이 있는지 찾아
18        matches = face_recognition.compare_faces(data["encodings"], encoding)
19        name = unknown_name # 'Unknown'으로 초기화
20
21        # check to see if we have found a match
22        if True in matches:
23            # 매칭된 pickle 파일의 True = encodings(특성값) 인덱스 번호를 찾아온다.
24            matchedIdxs = [i for (i, b) in enumerate(matches) if b]
25            counts = {} # 매칭된 얼굴별로 이름과 겹친 Count
26
27            for i in matchedIdxs:
28                # 매칭된 True = encodings(특성값) 인덱스 번호의 "names" value 를 찾아온다.
29                name = data["names"][i]
30                counts[name] = counts.get(name, 0) + 1 # counts dict의 "name" 키값의 value
31
32            # counts dict에서 key값(이름)들 중에서 가장 value 값(카운팅)이 큰 key값(이름)
33            name = max(counts, key=counts.get)
34
35            # names 배열에 이름을 추가
36            names.append(name)
37
38    # detection 된 얼굴의 boxing 위치 좌표와 이름 매칭
39    for ((top, right, bottom, left), name) in zip(boxes, names):
40        color = (0, 255, 0) # boxing color
41        line = 2 # boxing line 두께
42        if(name == unknown_name): # 만약 unknown 이라면
43            color = (0, 0, 255)
44            line = 1
45            name = '' # name 표시 안함
46
47        cv2.rectangle(image, (left, top), (right, bottom), color, line)
48        y = top - 15 if top - 15 > 15 else top + 15
49        cv2.putText(image, name, (left, y), cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, line)
50
51    end_time = time.time()
52    process_time = end_time - start_time
53    print("=== A frame took {:.3f} seconds".format(process_time))
54
55    # video 를 disk 에 output 하기 위해 writer 를 초기화한다.
56    global writer
57    if writer is None and output_name is not None:
58        fourcc = cv2.VideoWriter_fourcc(*"MJPG")
59        writer = cv2.VideoWriter(output_name, fourcc, 30,
60                                (image.shape[1], image.shape[0]), True)
61
62    # disk 에 frame 을 write 합니다.
63    if writer is not None:
64        writer.write(image)

```

■ 원본 동영상(1)에서 비디오 스트림을 읽고, 동영상 프레임 단위 얼굴 인식 및 결과를 출력

model=cnn ▶

```

1 # 원본 동영상(1)에서 video stream을 읽어온다.
2 cap = cv2.VideoCapture(file_name1)
3 writer = None
4 if not cap.isOpened():
5     print('--(!)Error opening video capture')
6     exit(0)
7
8 # 동영상 프레임 단위 얼굴 인식 및 결과 출력 반복
9 video_start_time = time.time() # 비디오 시작 시간
10 while True:
11     ret, frame = cap.read()
12     if frame is None:
13         # close the video file pointers
14         cap.release()
15         # close the writer point
16         writer.release()
17         print('--(!) No captured frame -- Break!')
18         break
19
20 # 얼굴 인식 및 결과 출력 함수 호출
21 detectAndDisplay(frame, output_name1)
22
23 video_end_time = time.time() # 비디오 종료 시간
24 video_process_time = video_end_time - video_start_time
25 print("=== A video took {:.3f} seconds".format(video_process_time))

```

```

=== A frame took 0.236 seconds
=== A frame took 0.190 seconds
=== A frame took 0.178 seconds
=== A frame took 0.176 seconds
=== A frame took 0.178 seconds
=== A frame took 0.179 seconds
=== A frame took 0.177 seconds
=== A frame took 0.178 seconds
=== A frame took 0.176 seconds
=== A frame took 0.174 seconds
=== A frame took 0.180 seconds
=== A frame took 0.177 seconds
=== A frame took 0.174 seconds
=== A frame took 0.177 seconds
=== A frame took 0.173 seconds
=== A frame took 0.177 seconds
=== A frame took 0.178 seconds
=== A frame took 0.176 seconds
=== A frame took 0.177 seconds
=== A frame took 0.175 seconds
=== A frame took 0.174 seconds
=== A frame took 0.185 seconds
=== A frame took 0.173 seconds
=== A frame took 0.178 seconds
=== A frame took 0.176 seconds
=== A frame took 0.174 seconds
=== A frame took 0.176 seconds
=== A frame took 0.174 seconds
=== A frame took 0.186 seconds
=== A frame took 0.175 seconds
=== A frame took 0.173 seconds
--(!) No captured frame -- Break!
=== A video took 6.044 seconds

```

실행 시간은 6.044 초가 소요

■ VM에서 출력 파일을 확인하고, 내 구글 드라이브로 이동

```
1 # 실행 결과 'output_cnn_chun_video1.mp4' 파일 확인
2
3 !ls
```

```
gdrive  output_cnn_chun_video1.mp4  output_cnn_chun_video2.mp4  sample_data
```

```
1 # 내 구글 드라이브로 파일 이동
2
3 !mv output_cnn_chun_video1.mp4 gdrive/My Drive/CV/Face Recognition/video/
4 print('output_cnn_chun_video1.mp4 move complete!!')
```

```
output_cnn_chun_video1.mp4 move complete!!
```

■ 원본 동영상(2)에서 비디오 스트림을 읽고, 동영상 프레임 단위 얼굴 인식 및 결과를 출력

model=cnn

```

1 # 원본 동영상(2)에서 video stream을 읽어온다.
2 cap = cv2.VideoCapture(file_name2)
3 writer = None
4 if not cap.isOpened():
5     print('--(!)Error opening video capture')
6     exit(0)
7
8 # 동영상 프레임 단위 얼굴 인식 및 결과 출력 반복
9 video_start_time = time.time() # 비디오 시작 시간
10 while True:
11     ret, frame = cap.read()
12     if frame is None:
13         # close the video file pointers
14         cap.release()
15         # close the writer point
16         writer.release()
17         print('--(!) No captured frame -- Break!')
18         break
19
20 # 얼굴 인식 및 결과 출력 함수 호출
21 detectAndDisplay(frame, output_name2)
22
23 video_end_time = time.time() # 비디오 종료 시간
24 video_process_time = video_end_time - video_start_time
25 print("=== A video took {:.3f} seconds".format(video_process_time))

```

```

=== A frame took 0.182 seconds
=== A frame took 0.185 seconds
=== A frame took 0.184 seconds
=== A frame took 0.181 seconds
=== A frame took 0.181 seconds
=== A frame took 0.186 seconds
=== A frame took 0.185 seconds
=== A frame took 0.183 seconds
=== A frame took 0.187 seconds
=== A frame took 0.186 seconds
=== A frame took 0.189 seconds
=== A frame took 0.216 seconds
=== A frame took 0.207 seconds
=== A frame took 0.211 seconds
=== A frame took 0.213 seconds
=== A frame took 0.206 seconds
=== A frame took 0.208 seconds
=== A frame took 0.213 seconds
=== A frame took 0.222 seconds
=== A frame took 0.211 seconds
=== A frame took 0.214 seconds
=== A frame took 0.207 seconds
=== A frame took 0.212 seconds
=== A frame took 0.208 seconds
=== A frame took 0.202 seconds
=== A frame took 0.205 seconds
=== A frame took 0.206 seconds
=== A frame took 0.216 seconds
--(!) No captured frame -- Break!
=== A video took 13.523 seconds

```

실행 시간은 13.523 초가 소요

■ VM에서 출력 파일을 확인하고, 내 구글 드라이브로 이동

```
1  # 실행 결과 'output_cnn_chun_video2.mp4' 파일 확인
2
3  !ls
```

```
gdrive  output_cnn_chun_video2.mp4  sample_data
```

```
1  # 내 구글 드라이브로 파일 이동
2
3  !mv output_cnn_chun_video2.mp4 gdrive/My Drive/CV/Face Recognition/video/
4  print('output_cnn_chun_video2.mp4 move complete!!')
```

```
output_cnn_chun_video2.mp4 move complete!!
```

원본 동영상과 얼굴 특성값 파일 위치, 이미지에서 주요 특징을 캡처하는 표현 방법을 “hog”으로 설정

주요 특징 캡처하는 시간
: 조금 빠름

정확도 : 낮음

```

1  # 원본 동영상과 얼굴 특성값 저장 위치 설정
2  # - 이미지에서 주요 특징을 캡처하는 표현 방법 : hog
3  # - hog : 조금 빠르지만 정확도가 낮음
4
5  file_name1 = 'gdrive/My Drive/CV/Face Recognition/video/chun_video1.mp4' # 원본 동영상
6  file_name2 = 'gdrive/My Drive/CV/Face Recognition/video/chun_video2.mp4' # 원본 동영상
7  encoding_file = 'gdrive/My Drive/CV/Face Recognition/encodings.pickle' # 저장해 두었던 특성 파일
8  unknown_name = 'Unknown' # 이름을 모를 경우 Unknown 으로 지정
9  output_name1 = 'output_hog_chun_video1.mp4' # Detection 된 output 동영상
10 output_name2 = 'output_hog_chun_video2.mp4' # Detection 된 output 동영상
11
12 # Either cnn or hog. The CNN method is more accurate but slower. HOG is faster but less accurate.
13 model_method = 'hog'
    
```

■ 원본 동영상(1)에서 비디오 스트림을 읽고, 동영상 프레임 단위 얼굴 인식 및 결과를 출력

model=hog ▶

```

1 # 원본 동영상(1)에서 video stream을 읽어온다.
2 cap = cv2.VideoCapture(file_name1)
3 writer = None
4 if not cap.isOpened():
5     print('--(!)Error opening video capture')
6     exit(0)
7
8 # 동영상 프레임 단위 얼굴 인식 및 결과 출력 반복
9 video_start_time = time.time() # 비디오 시작 시간
10 while True:
11     ret, frame = cap.read()
12     if frame is None:
13         # close the video file pointers
14         cap.release()
15         # close the writer point
16         writer.release()
17         print('--(!) No captured frame -- Break!')
18         break
19
20 # 얼굴 인식 및 결과 출력 함수 호출
21 detectAndDisplay(frame, output_name1)
22
23 video_end_time = time.time() # 비디오 종료 시간
24 video_process_time = video_end_time - video_start_time
25 print("=== A video took {:.3f} seconds".format(video_process_time))

```

```

=== A frame took 0.644 seconds
=== A frame took 0.669 seconds
=== A frame took 0.674 seconds
=== A frame took 0.651 seconds
=== A frame took 0.647 seconds
=== A frame took 0.664 seconds
=== A frame took 0.648 seconds
=== A frame took 0.659 seconds
=== A frame took 0.669 seconds
=== A frame took 0.655 seconds
=== A frame took 0.655 seconds
=== A frame took 0.666 seconds
=== A frame took 0.651 seconds
=== A frame took 0.664 seconds
=== A frame took 0.724 seconds
=== A frame took 1.014 seconds
=== A frame took 0.999 seconds
=== A frame took 1.009 seconds
=== A frame took 1.004 seconds
=== A frame took 0.726 seconds
=== A frame took 0.658 seconds
=== A frame took 0.663 seconds
=== A frame took 0.642 seconds
=== A frame took 0.662 seconds
=== A frame took 0.634 seconds
=== A frame took 0.639 seconds
=== A frame took 0.648 seconds
=== A frame took 0.649 seconds
=== A frame took 0.640 seconds
=== A frame took 0.652 seconds
=== A frame took 0.621 seconds
--(!) No captured frame -- Break!
=== A video took 22.330 seconds

```

실행 시간은 22.330 초가 소요

■ VM에서 출력 파일을 확인하고, 내 구글 드라이브로 이동

```

1  # 실행 결과 'output_hog_chun_video1.mp4' 파일 확인
2
3  !ls

gdrive  output_hog_chun_video1.mp4  sample_data

1  # 내 구글 드라이브로 파일 이동
2
3  !mv output_hog_chun_video1.mp4 gdrive/My Drive/CV/Face Recognition/video/
4  print('output_hog_chun_video1.mp4 move complete!!')

output_hog_chun_video1.mp4 move complete!!

```

■ 원본 동영상(2)에서 비디오 스트림을 읽고, 동영상 프레임 단위 얼굴 인식 및 결과를 출력

model=hog

```

1 # 원본 동영상(2)에서 video stream을 읽어온다.
2 cap = cv2.VideoCapture(file_name2)
3 writer = None
4 if not cap.isOpened():
5     print('--(!)Error opening video capture')
6     exit(0)
7
8 # 동영상 프레임 단위 얼굴 인식 및 결과 출력 반복
9 video_start_time = time.time() # 비디오 시작 시간
10 while True:
11     ret, frame = cap.read()
12     if frame is None:
13         # close the video file pointers
14         cap.release()
15         # close the writer point
16         writer.release()
17         print('--(!) No captured frame -- Break!')
18         break
19
20 # 얼굴 인식 및 결과 출력 함수 호출
21 detectAndDisplay(frame, output_name2)
22
23 video_end_time = time.time() # 비디오 종료 시간
24 video_process_time = video_end_time - video_start_time
25 print("=== A video took {:.3f} seconds".format(video_process_time))

```

```

=== A frame took 0.004 seconds
=== A frame took 0.663 seconds
=== A frame took 0.648 seconds
=== A frame took 0.668 seconds
=== A frame took 0.649 seconds
=== A frame took 0.972 seconds
=== A frame took 0.980 seconds
=== A frame took 1.001 seconds
=== A frame took 1.020 seconds
=== A frame took 0.876 seconds
=== A frame took 0.651 seconds
=== A frame took 0.665 seconds
=== A frame took 0.648 seconds
=== A frame took 0.657 seconds
=== A frame took 0.658 seconds
=== A frame took 0.628 seconds
=== A frame took 0.662 seconds
=== A frame took 0.640 seconds
=== A frame took 0.658 seconds
=== A frame took 0.657 seconds
=== A frame took 0.670 seconds
=== A frame took 0.643 seconds
=== A frame took 0.644 seconds
=== A frame took 0.635 seconds
=== A frame took 0.771 seconds
=== A frame took 1.010 seconds
--(!) No captured frame -- Break!
=== A video took 47.980 seconds

```

실행 시간은 47.980 초가 소요

■ VM에서 출력 파일을 확인하고, 내 구글 드라이브로 이동

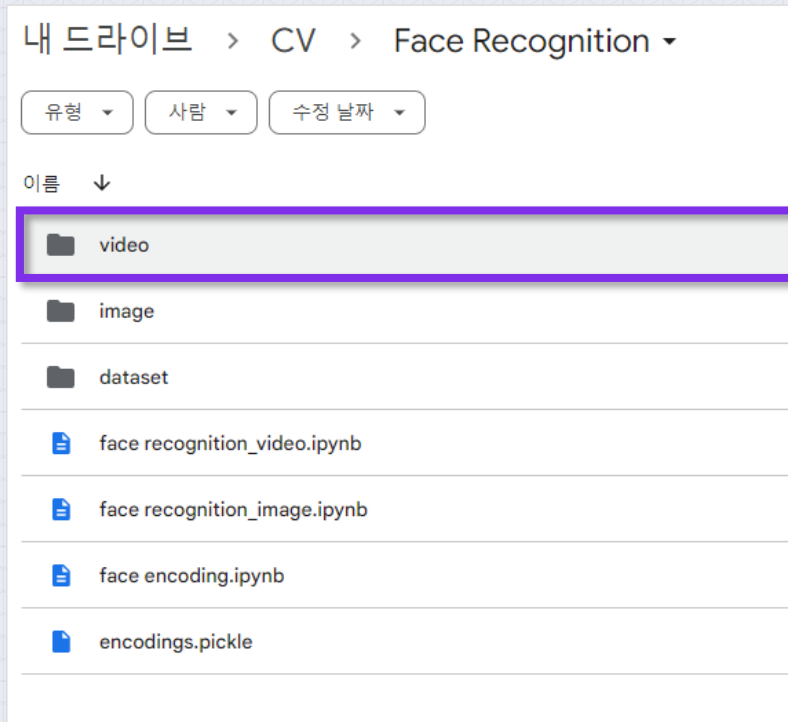
```
1  # 실행 결과 'output_hog_chun_video2.mp4' 파일 확인
2
3  !ls
```

```
gdrive  output_hog_chun_video2.mp4  sample_data
```

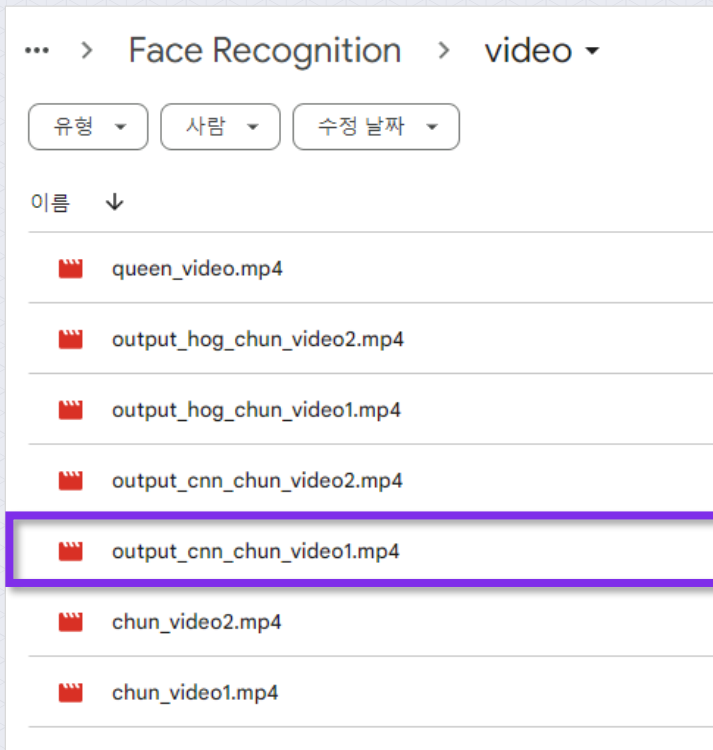
```
1  # 내 구글 드라이브로 파일 이동
2
3  !mv output_hog_chun_video2.mp4 gdrive/My Drive/CV/Face Recognition/video/
4  print('output_hog_chun_video2.mp4 move complete!!')
```

```
output_hog_chun_video2.mp4 move complete!!
```

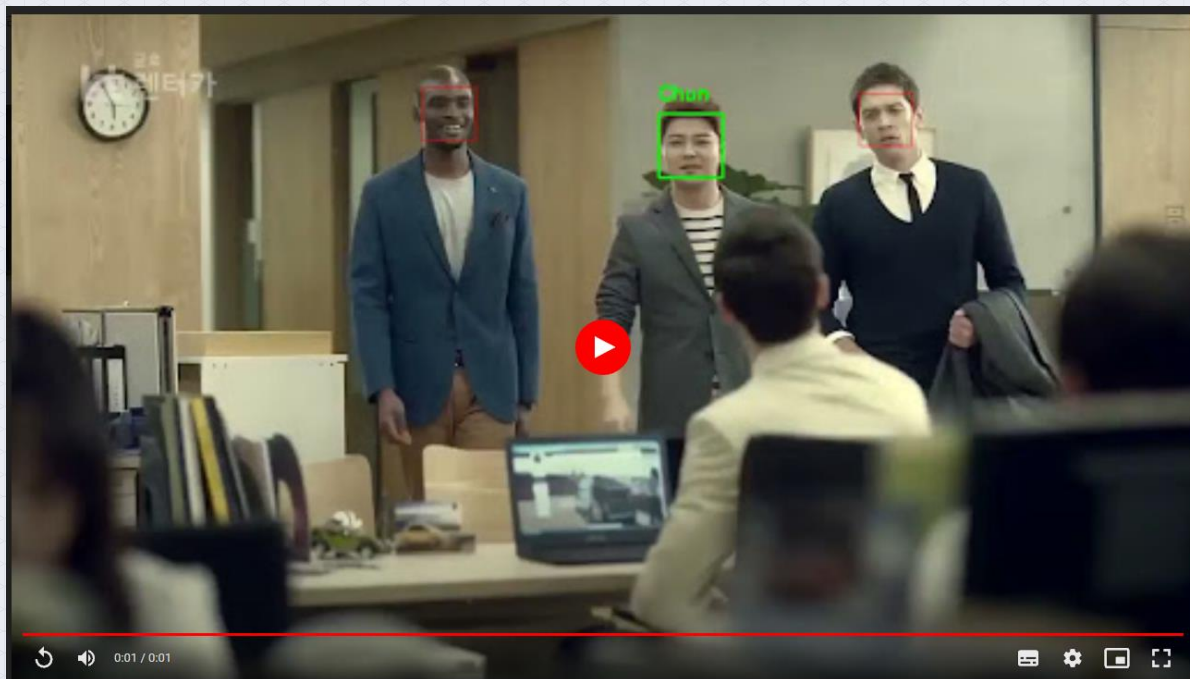

■ CV < Face Recognition < video 폴더 클릭



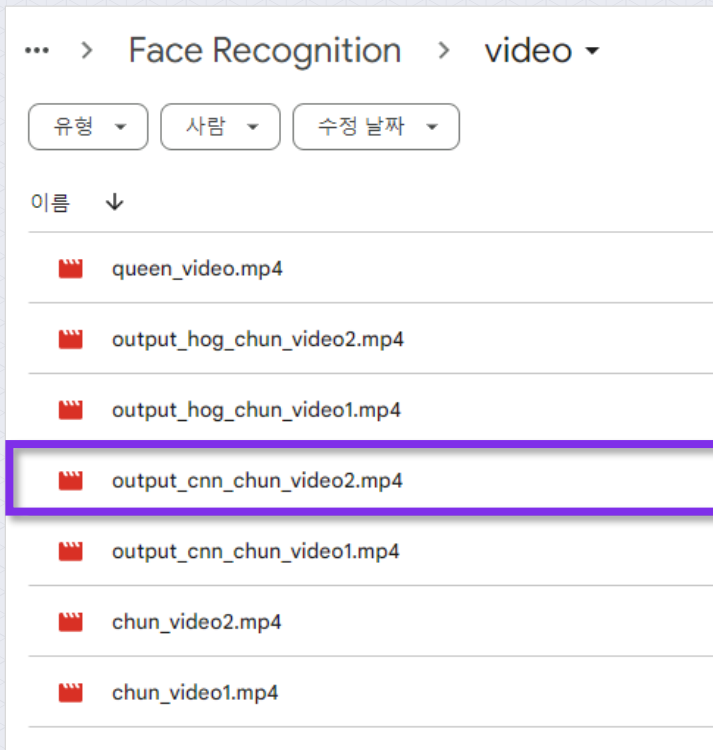
■ CV < Face Recognition < video < output_cnn_chun_video1.mp4 파일 클릭



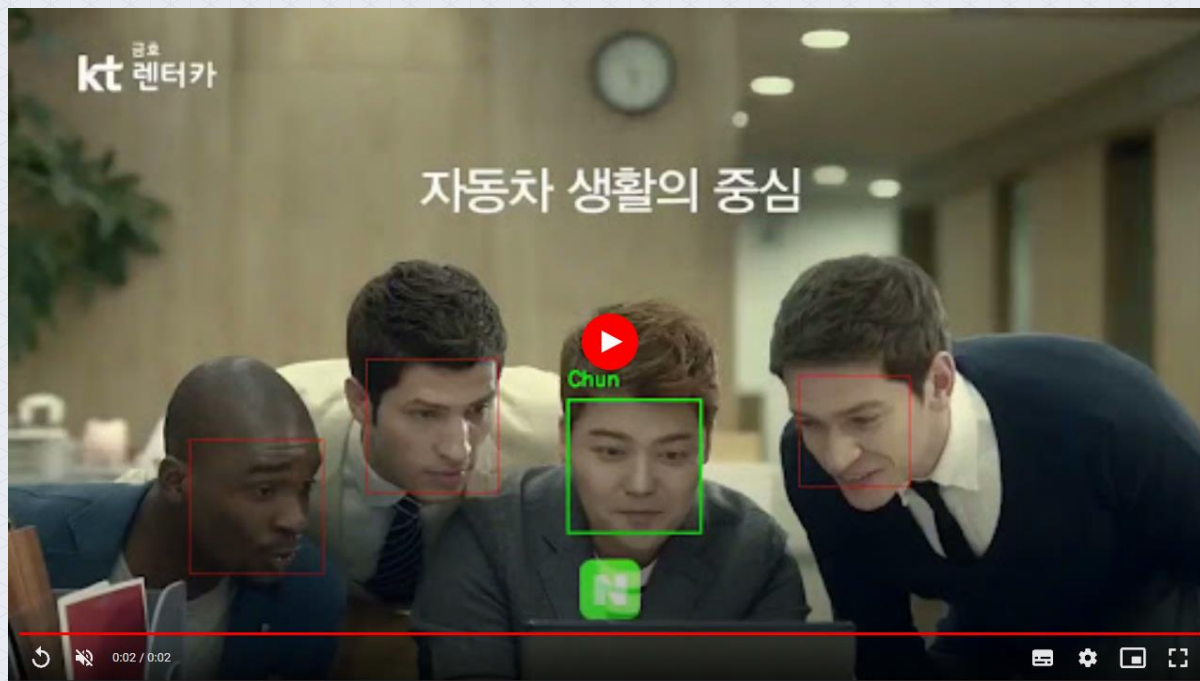
- 출력 파일에서 전현무 방송인의 얼굴 인식이 잘 된 것을 볼 수 있음



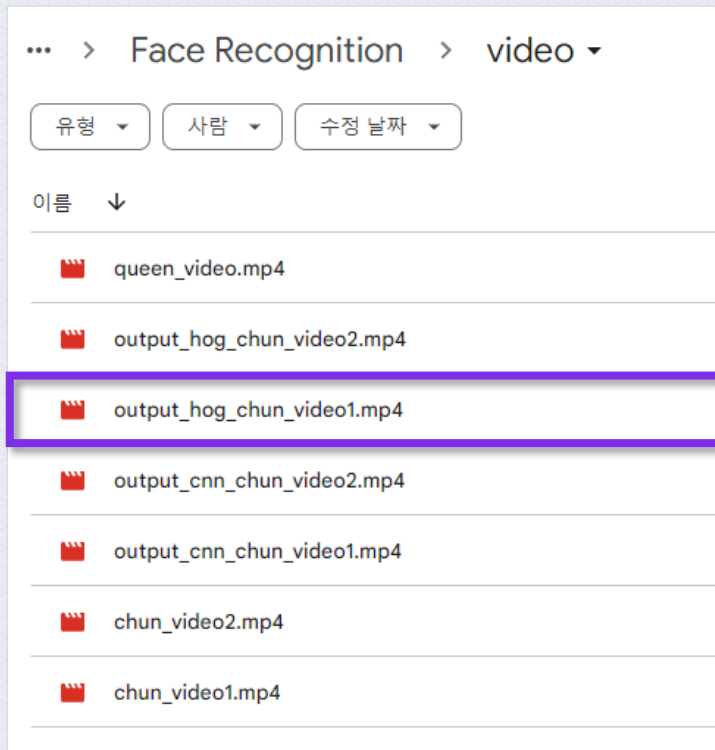
■ CV < Face Recognition < video < output_cnn_chun_video2.mp4 파일 클릭



- 출력 파일에서 전현무 방송인의 얼굴 인식이 잘 된 것을 볼 수 있음



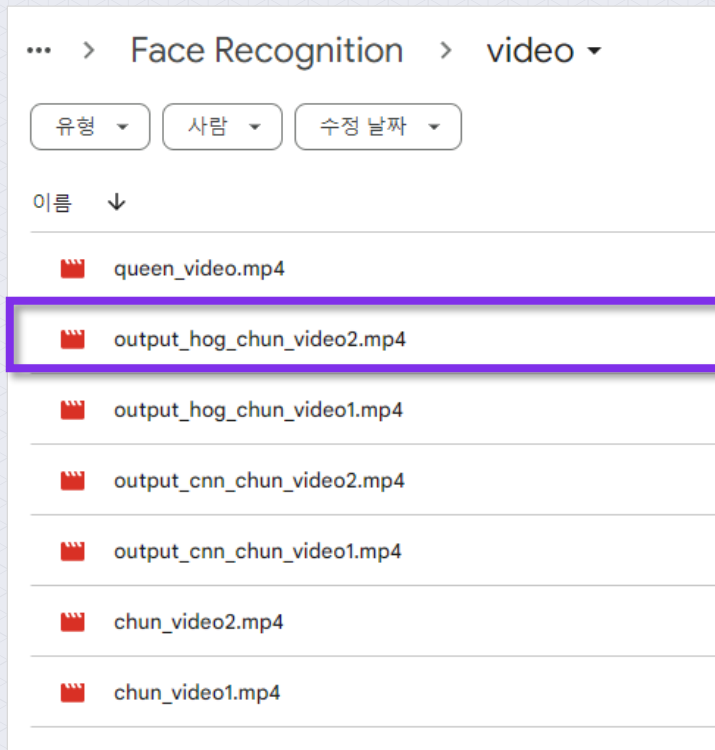
■ CV < Face Recognition < video < output_hog_chun_video1.mp4 파일 클릭



- 출력 파일에서 전현무 방송인의 얼굴 인식이 잘 된 것을 볼 수 있음



■ CV < Face Recognition < video < output_hog_chun_video2.mp4 파일 클릭



- 출력 파일에서 전현무 방송인의 얼굴 인식이 잘 된 것을 볼 수 있음

